

# Late Payment Policy

A verified payment can arrive after the checkout expired. The money moved whatever our timer did, so it is never ignored: the order ends as **paid** when fulfilling is still safe, and as **refund\_due** when it is not. No confirmed payment is invisible any more.

Branch **claude/create-app-repository-1cxsih**

Code state **4342c06**

Documentation-only since

Tests **349 passed · 24 browser**

Mutations **8 of 8 caught**

PROVENANCE – WHAT THIS DOCUMENT DESCRIBES, EXACTLY

**Branch:** claude/create-app-repository-1cxsih. Not merged into main.

**Code state under review: 4342c06** — the last commit that touches implementation. Everything after it on this branch is documentation, and that is checkable in one command:

```
git diff 4342c06 HEAD -- '!:docs'  # returns nothing
```

**Every commit after 4342c06 on this branch is documentation.** That is the whole reason this packet leads with a code state rather than a branch tip: the tip moves each time the packet itself is revised, and the code does not. CI has been green on the code state and on every documentation commit since — runs #23, #24 and #25.

| COMMIT          | WHAT IT IS                                                                                                                                                    | TOUCHES CODE | CI                                  |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|-------------------------------------|
| 5d92c8e         | The policy: state machine, grace window, row lock, commerce reconciliation, migration 0011, 31 tests, architecture doc                                        | yes          | <div>#22 RED</div> browser race, §7 |
| 4342c06         | The fix for that race. <b>The last commit touching code.</b>                                                                                                  | yes          | <div>#23 GREEN</div>                |
| 9890cdf         | This review packet, first issue                                                                                                                               | no           | <div>#24 GREEN</div>                |
| ec12792         | Packet reissued with a provenance block                                                                                                                       | no           | <div>#25 GREEN</div>                |
| ...and this one | Packet reissued again so that no future documentation commit can make it read as stale. <b>The list below the code state may grow; the code state cannot.</b> | no           | runs on push                        |

#### WHY THE HEAD SHOWN IS THE CODE STATE AND NOT THE TIP

A document cannot contain the hash of the commit that adds it — writing the hash changes the content, which changes the hash. Any packet that names the branch tip is therefore stale by exactly one commit the moment it is committed, and the previous issue of this packet was stale for precisely that reason.

So the identifier this packet leads with is the one that *stops moving*: the code state. It has not changed since 4342c06, the command above proves it in one line, and every commit after it is listed in the table with what it contains. Nothing about the implementation under review is ambiguous.

**This document names no mutable identifier at all**, so no number of further documentation commits can make it wrong. If a single immutable hash is wanted for the sign-off record, the right instrument is a git tag on the final commit — say so and one will be created; a document cannot be that instrument.

1

## What happened before, traced rather than remembered

The investigation ran the real domain code against a real database. This is what it printed:

```
after initiation    : order=pending_payment payment=pending  entitlements=0 fee_schedules=0
after expiry       : order=expired          payment=pending  entitlements=0 fee_schedules=0
webhook outcome    : {"outcome":"applied", "orderStatus":"expired"}
after late confirm : order=expired          payment=succeeded entitlements=0 fee_schedules=0
reconciliation clean : true
ledger entries     : 0
```

#### THREE FINDINGS, NONE OF THEM COMFORTABLE

**The money left no commercial trace.** The payment moved because every check passed. The order's guarded `UPDATE` named `pending_payment`, matched nothing, and the entitlement and fee-schedule writes sat inside that branch.

**Nothing surfaced it.** Reconciliation compared ledger balances and reported clean, because it knew nothing about orders and the ledger was empty by design. The only trace was one audit row.

**The buyer could be charged twice.** Repeat purchase is refused by checking `entitlements`. A late payment created none, so the second checkout succeeded — reachable, undetected, and the buyer's only remedy was a support ticket.

The blueprint could not settle it: its order machine is `created → awaiting_payment → paid → fulfilled → settled` and has no `expired` state at all. The checkout TTL arrived later. So the question went to review, and came back as option D.

| WHEN THE PAYMENT IS VERIFIED                     | CONDITIONS                                                          | ORDER BECOMES                                    |
|--------------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------|
| While the order is <code>pending_payment</code>  | the ordinary path, unchanged                                        | <code>PAID</code> entitlement + frozen fees      |
| After expiry, at or before expiry + 24 h         | product published · store published · buyer does not already own it | <code>PAID</code> + <code>late_payment_at</code> |
| After expiry, more than 24 h later               | —                                                                   | <code>REFUND_DUE</code>                          |
| After expiry, inside 24 h, any condition failing | —                                                                   | <code>REFUND_DUE</code>                          |
| A failed payment after expiry                    | no money arrived                                                    | <code>STAYS_EXPIRED</code>                       |

#### WHAT `REFUND_DUE` MEANS, IN THE WORDS USED EVERYWHERE

“Payment successfully confirmed, but Afrinext did not fulfil the order and must later execute a refund through the appropriate payment provider.”

It is a **queue and a state**. It is not an executed refund, not a request sent to anybody, and not a credit. **No refund execution exists in the codebase**, and nothing drains this queue. The buyer's screen says exactly that, in French and English, and stops there.

### The boundary is inclusive, and tested at the millisecond

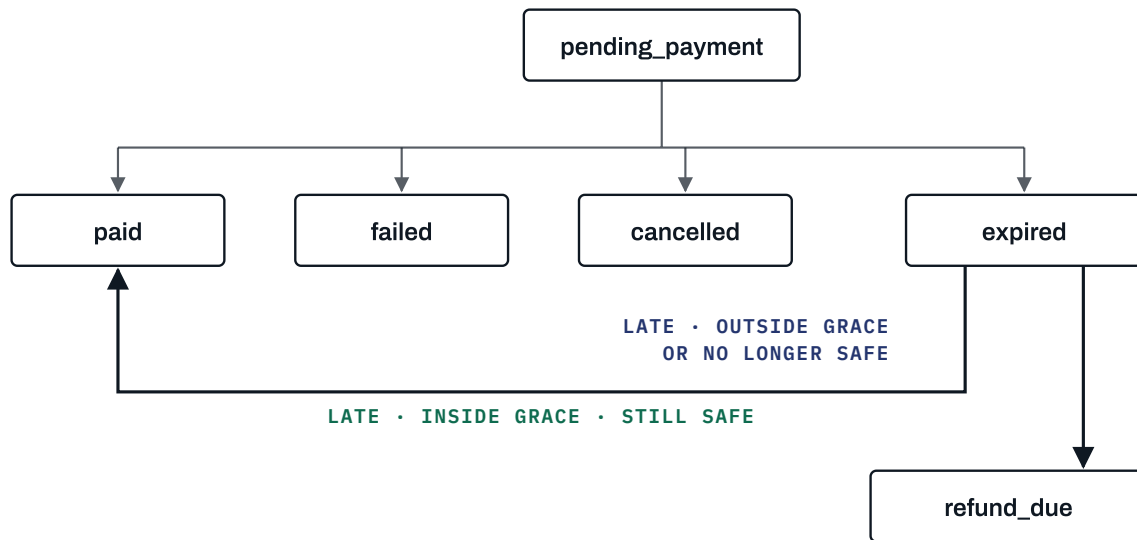
Grace applies when the payment is verified at or before `expires_at + 24 h`. One millisecond later is `refund_due`. That is why `applyProviderEvent` now takes a clock: a boundary nobody can land on is a boundary nobody has checked. Two tests land on it — exactly at, and exactly one millisecond past.

### Why 24 hours

Mobile-money confirmations in Niger travel through a carrier and an aggregator before they reach Afrinext. A delay of seconds is ordinary; a delay of hours happens. Refusing those would take money from people who genuinely paid and hand every one of them to support, for no benefit to anybody. Beyond a day the balance tips: the buyer has moved on, may have bought again, and fulfilment becomes a surprise rather than a service.

| SETTING                                          | VALUE         | HOW IT IS READ                                                                                                                                                                                                                                     |
|--------------------------------------------------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>checkout.ttl_seconds</code>                | 1800 · 30 min | Seeded explicitly. The code constant is now a fallback, not the source.                                                                                                                                                                            |
| <code>checkout.late_payment_grace_seconds</code> | 86400 · 24 h  | A <b>ceiling, not a default</b> . The loader clamps to it, so an operator may narrow the window to zero without asking anyone; widening it is a code change and a review. A malformed value falls back to 24 h — never to zero, never to no limit. |

The asymmetry is deliberate and tested: a stored `99999999` still yields 24 hours, and a stored `null` , `"soon"` , `-1` or an object yields 24 hours rather than the zero a coercing read would have produced.



expired is the ONLY state an order can leave – money can arrive after it. Every other state is terminal.

The two thick arrows are new. **Payments are unchanged:** `initiated → pending → succeeded | failed | cancelled | expired`, each terminal.

Both new transitions are guarded `UPDATE` s naming `status = 'expired'`, and the `CHECK` constraint on `orders.status` was widened to include `refund_due` – a test writes `'refunded'` and requires the database to refuse it.

4

## Financial invariants

1

### No successful payment is invisible

Every `succeeded` payment leaves its order in `paid` or `refund_due`. Nothing else is a legal resting place, and reconciliation looks for violations.

2

### No double fulfilment

One entitlement per buyer per product, by unique index. A late payment for something already owned becomes `refund_due` rather than being absorbed.

3

### No double charge swallowed in silence

If a buyer paid twice, the second payment is queued and countable rather than consumed.

4

### Fee snapshots are unchanged

A late-accepted order freezes its commission through the *same function* as an on-time one — the two paths were merged into one `fulfil()` precisely so they cannot drift apart.

5

### The ledger is untouched

Neither acceptance nor `refund_due` posts a ledger entry. Payment confirmation and financial settlement remain separate, and settlement is still not implemented.

## Concurrency: the row lock, and the test that needs it

The confirmation transaction takes `SELECT ... FOR UPDATE` on the order row **before deciding anything**. A confirmation and the expiry sweeper queue rather than interleave: whichever arrives second sees the state the first committed and takes the branch that state deserves.

### PROVED DETERMINISTICALLY, NOT BY HOPING TWO PROMISES INTERLEAVE

A test opens a **second connection**, takes `for update` on the order, and holds it. The confirmation arrives and blocks. The holder then expires the order and commits. Only then does the confirmation get to look — and it sees `expired`, takes the late path, and accepts.

**Without the lock that test fails**, and it fails in the exact shape this gate exists to abolish: the confirmation reads the pre-expiry snapshot, takes the ordinary path, and its guarded `UPDATE` then matches nothing — payment succeeded, order expired, orphaned.

Ledger reconciliation asks whether the books balance. It could not ask whether a succeeded payment left its order somewhere sensible, because it knows nothing about orders — and that gap is how the original defect stayed silent. `reconcileCommerce()` is the missing half:

| IMPOSSIBLE STATE            | WHY IT MATTERS                                                                                             |
|-----------------------------|------------------------------------------------------------------------------------------------------------|
| succeeded_payment_orphaned  | Money arrived and left no commercial trace. The original defect.                                           |
| paid_order_without_payment  | Fulfilment without money — the mirror image, and worse for the seller.                                     |
| multiple_succeeded_payments | The buyer was charged twice. The partial unique index bounds live attempts, not successful ones over time. |
| refund_due_without_payment  | Either the queue is wrong or the payment record is.                                                        |
| duplicate_entitlement       | Unreachable today; the check is what notices if that unique index is ever dropped.                         |

#### THE GATE PROVES IT CAN STILL SPEAK

CI asserts the result is empty — but a detector that reports “clean” is worth nothing unless it can report something else, and the suite truncates, so on some runs there would be no orders left and the assertion would pass *vacuously*. That is exactly the failure this gate was opened for.

So on every run the script builds one consistent order-and-payment pair, requires silence, breaks one invariant, **requires the detector to name it**, and puts it back. A run that cannot demonstrate detection fails the build.



```

pnpm lint                exit 0
pnpm typecheck            exit 0
pnpm test                 349 passed | 3 skipped (22 files)   was 317
pnpm build                exit 0
playwright test           24 passed    production mode, DB from zero
ledger concurrency        6 passed
reconcile-check           ledger clean · commerce clean · detector proved
pnpm audit --audit-level high No known vulnerabilities found
mutation testing           8 of 8 caught

```

**31 new tests** in `orders/late-payment.test.ts`, covering all fourteen requested scenarios plus the configuration and state-machine cases, and **2 new tests** in the delivery suite for the two halves of the decision.

CI

**CI:** run #23 (32414854416) on the code state 4342c06 — **ALL THREE JOBS GREEN** — and green again on every documentation commit since: #24 (32415407611) and #25 (32420898168). Run #22 on 5d92c8e was **RED** on the browser job: a pre-existing race in the checkout test, described below and fixed in 4342c06. `lint · typecheck · test` passed on all three.

| DELIBERATE DEFECT                                                    | RESULT            |
|----------------------------------------------------------------------|-------------------|
| The grace-window check is removed                                    | CAUGHT · 8 TESTS  |
| The <code>expired → refund_due</code> branch is removed              | CAUGHT · 10 TESTS |
| The already-owned check is removed                                   | CAUGHT            |
| <code>SELECT ... FOR UPDATE</code> is removed                        | CAUGHT            |
| The grace clamp is removed, so settings can widen the policy         | CAUGHT            |
| The orphaned-payment reconciliation condition is loosened            | CAUGHT            |
| The duplicate-payment reconciliation condition is loosened           | CAUGHT            |
| The entitlement <code>on conflict do nothing</code> guard is removed | CAUGHT            |

#### A SECOND COMMIT: CI FOUND A RACE IN A TEST, NOT IN THE POLICY

The browser job failed on the checkout journey while every local run passed. The test waited for `pay-waiting` — static text inside the form, on screen before the server action has done anything — and then read the payment row, which did not exist yet. Local timing won that race; CI lost it.

The race predates this gate and has nothing to do with the late-payment policy. The order page now shows the charge's real status, which appears only once the action has completed, and the test waits for *that*. It is also better for the buyer: “I have not paid yet” and “the provider has not answered yet” looked identical before.

#### THE ROW-LOCK MUTATION ESCAPED AT FIRST, AND THE FIX IS THE INTERESTING PART

Removing `FOR UPDATE` changed nothing, because the concurrency test used `Promise.all` and two promises do not reliably interleave two transactions — it created a *possible* race, not a certain one. A test that passes whether or not the lock exists is not testing the lock.

The replacement holds the row lock from a second connection and controls the ordering exactly. It now fails without the lock, every time.

## Files changed — 21 across the gate

| FILE                                     | WHY                                                                                                                                                    |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>The policy</b>                        |                                                                                                                                                        |
| core/orders/state.ts                     | <code>refund_due</code> added; <code>expired</code> gains two exits and stops being terminal.                                                          |
| core/orders/payment.ts                   | The row lock, the late decision and its four conditions, the shared <code>fulfil()</code> , the injected clock, the decision written to the audit log. |
| core/orders/checkout.ts                  | <code>loadLatePaymentGraceSeconds()</code> — clamped to the reviewed maximum, malformed values fall back rather than widen.                            |
| <b>Reconciliation</b>                    |                                                                                                                                                        |
| core/ledger/commerce-reconcile.ts        | New. The five impossible commerce states.                                                                                                              |
| core/scripts/reconcile-check.ts          | The CI gate, plus the probe that proves the detector can still report a fault.                                                                         |
| core/ledger/index.ts                     | Export.                                                                                                                                                |
| <b>Schema and configuration</b>          |                                                                                                                                                        |
| db/migrations/0011 + meta/_journal.json  | Widened status CHECK, <code>late_payment_at</code> and its constraint, the refund-queue index, both settings.                                          |
| db/schema/commerce.ts                    | The same, in Drizzle.                                                                                                                                  |
| core/seed/reference.ts                   | <code>checkout.ttl_seconds</code> and <code>checkout.late_payment_grace_seconds</code> seeded explicitly.                                              |
| <b>Tests</b>                             |                                                                                                                                                        |
| core/orders/late-payment.test.ts         | New. 31 tests: all fourteen requested scenarios, the configuration cases, the state machine, and the five reconciliation detections.                   |
| core/orders/orders.test.ts               | Two tests updated to the approved policy — named below.                                                                                                |
| core/content/content.test.ts             | One updated, one added, for the two halves of the decision.                                                                                            |
| web/e2e/checkout.spec.ts                 | Waits for the payment state instead of a static caption — the CI race.                                                                                 |
| <b>Web and documentation</b>             |                                                                                                                                                        |
| web/[locale]/orders/[id]/page.tsx        | Shows the charge's real status, and states a refund is owed <i>and not executed</i> .                                                                  |
| i18n fr.json · en.json                   | 4 keys per locale, worded so nothing claims a refund happened.                                                                                         |
| docs/architecture/late-payment-policy.md | New. The decision, the table, the machine, the invariants, the settings, and what is not built.                                                        |
| AGENTS.md                                | The rule added to the non-negotiables.                                                                                                                 |
| docs/review/... .html · .pdf             | This packet.                                                                                                                                           |

No new dependency; `pnpm-lock.yaml` is untouched. `ledger/flows.ts`, `ledger/post.ts`, `ledger/accounts.ts`, `money/`, `authz/`, `consent/`, `auth/` and `payments/` have no diff.

### Three existing tests were updated, and why that is not a weakening

#### NAMED HERE RATHER THAN LEFT IN THE DIFF

`orders.test.ts` · “lets an order leave `pending_payment` exactly once” asserted every non-pending state was terminal. `expired` no longer is. It now asserts the new machine *exactly*: expired reaches `paid` and `refund_due` and nothing else, and every other state is still terminal — more specific than before, not less.

`orders.test.ts` · “refuses a confirmation arriving after the order expired” asserted the behaviour this gate replaced. It keeps the invariant that matters — a succeeded payment never leaves its order in limbo — and points at the new suite for the edges.

`content.test.ts` · “grants nothing when the order expired” now uses a payment past the grace window, so it still proves that a `refund_due` order grants no content. A second test was added for the other half: a payment a minute late *does* deliver the file.

## Risks, and what is deliberately not built

| CONFIRMATION                                                                                                                                                                                     |           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| No refund execution, through any provider                                                                                                                                                        | CONFIRMED |
| No iPayMoney code, credentials, endpoints, payloads or signatures. The mock remains the only provider, and nothing in this policy is provider-specific                                           | CONFIRMED |
| No settlement, seller wallet or payout                                                                                                                                                           | CONFIRMED |
| No ledger architecture change. <code>ledger/flows</code> , <code>ledger/post</code> , <code>ledger/accounts</code> and <code>money/</code> have no diff; entitlement and refund_due post nothing | CONFIRMED |
| Webhook, payment and entitlement idempotency preserved and re-tested under the new paths                                                                                                         | CONFIRMED |
| Authorization, consent, OTP and pricing behaviour untouched                                                                                                                                      | CONFIRMED |
| No new dependency                                                                                                                                                                                | CONFIRMED |

### Open risks

| ITEM                                                                                                                                                                                                                                                | STATE                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| Nothing drains the <code>refund_due</code> queue. The state is correct, countable and visible to the buyer; the money still has to be returned by a human, out of band, until refund execution exists. <b>That is the next thing I would build.</b> | NEXT                    |
| No operator screen lists the queue. Finding it today is a SQL query or the reconciliation report. Nobody is alerted.                                                                                                                                | OPEN                    |
| The buyer is not notified. Their order page says a refund is owed and not yet executed; nothing pushes that to them, because no messaging provider is chosen.                                                                                       | BLOCKED ON SMS PROVIDER |
| Reconciliation is run by CI, not on a schedule. In production it needs a job and an alarm; neither exists.                                                                                                                                          | BEFORE LAUNCH           |
| A late acceptance re-opens a closed order, which is a real change in behaviour a seller may notice as a sale appearing a day later. Deliberate, and the reason the window is a day rather than a week.                                              | ACCEPTED                |
| The grace window is measured from <code>expires_at</code> , not from when the buyer left for the provider. An order that expired early for another reason inherits the same window.                                                                 | ACCEPTED                |

Is Phase 2 ready for approval? The financial policy gate is closed: the rule is decided, documented, implemented, enforced by guarded transitions and a row lock, and watched by a reconciliation that proves it can still report a fault. What is *not* closed is the consequence of that rule — a refund queue with no drain. My recommendation is to approve the gate and treat refund execution as the first item of the next milestone, before settlement, because it is the only open item that can leave a real buyer out of pocket.

---

PROVENANCE, REPEATED – IDENTICAL TO PAGE ONE

**Branch:** `claude/create-app-repository-1cxsih` · not merged into main.

**Code state under review:** `4342c06` · CI run #23 green.

**Documentation-only since:** `4342c06` · CI green on every commit after it — runs #24, #25.

**Gate commits:** `5d92c8e` → `4342c06` → `9890cdf` → `ec12792` → this reissue.

**Proof the packet is not stale:** `git diff 4342c06 HEAD -- ':!docs'` returns nothing.

**Final results:** 349 domain tests · 24 browser tests · 8 of 8 mutations caught · ledger and commerce reconciliation clean · dependency audit clean.

Phase 2 · financial review gate · branch `claude/create-app-repository-1cxsih` · code state `4342c06` · documentation-only since